# Take-Home Assignment - 1

## PDF Data Extraction for Legal Documents

**Duration** 1-2 Days

*Confidential - For evaluation purposes only*

---

## 1. Overview

Build a generic extraction system that pulls a **single piece of structured information** from a legal PDF document per call. The system accepts a natural-language query describing one term to extract, an optional set of few-shot examples, and an expected output type (string, date, number, or an array of these).

Each invocation extracts **exactly one term** - for example, the tenant name, or the termination notice period, or whether a non-compete clause exists. To extract multiple terms from the same document, the caller makes multiple calls.

For test data, you may use the CUAD (Contract Understanding Atticus Dataset) or any other publicly available legal document dataset.

---

## 2. Interface

Implement a function (or CLI command) with this signature:

```
None
extract(
    pdf: bytes | path,          # The source PDF document
    query: str,                 # Single term to extract (natural language)
    output_type: ...,           # Expected answer type (string, date, number, array, etc.)
    examples: list[dict] = None # Optional few-shot examples
) -> dict                       # See response format below
```

> The output_type parameter is intentionally flexible - you decide how to represent it (an enum, a string like "array[date]", a JSON Schema snippet, etc.).

### Response Format

Every call must return a structured response containing:

```
JSON
{
  "value": "...",
  "found": true,
  "sources": [{
    "page": 3,
    "snippet": "...the Tenant shall provide 60 days written
notice..."
  }]
}
```

- **value** - The extracted answer, conforming to the output schema.
- **found** - Boolean indicating whether the term was located in the document.
- **sources** - The page number and the relevant text snippet(s) from the PDF that supports the extracted value. This is critical for auditability in the legal domain.

### Supported Answer Types

| Type | Description | Example Value |
|---|---|---|
| String | Free-form text, names, clauses | "Acme Corp" |
| Date | ISO-8601 date string | "2024-03-15" |
| Number | Numeric values (integers or decimals) | 5.25 |
| Array of Strings | Multiple text values | ["Party A", "Party B"] |
| Array of Dates | Multiple dates | ["2024-01-01", "2025-01-01"] |
| Array of Numbers | Multiple numeric values | [5.25, 3.50] |

The output schema should be kept simple and you are free to define how schemas are represented (JSON Schema, a lightweight type hint, an enum, etc.) as long as the caller can express the expected answer type and the system enforces it.

---

# 3. Requirements

## 3.1 PDF Ingestion

- Accept PDF files via file path or raw bytes.
- Handle multi-page text-based PDF documents.
- Preserve structure where possible: tables, numbered clauses, section headers.

## 3.2 Query + Type Enforcement

- Interpret the natural-language query as a single term to extract and use the output_type to enforce the answer type.
- Validate the result type before returning (e.g., if the caller asks for number, don't return a string).
- Return a clear "found": false indicator when the term cannot be located

### 3.3 Source Attribution (Mandatory)

- Every extracted value **must** include the page number and a relevant text snippets from the source PDF.
- The snippets should be the actual passage the answer was derived from, not a paraphrase

### 3.4 Few-Shot Examples

When examples are provided (each with an "input" and "output" key), incorporate them into your prompt to improve extraction accuracy.

### 3.5 General

- Keep components modular: PDF parser, LLM client, and type validator should be separate.
- Return structured errors - never let raw LLM exceptions reach the caller.
- Be cost-conscious with token usage: avoid sending entire documents when targeted sections suffice.

---

## 4. Output Types & Examples

Each schema tells the system what type of answer to return. How you represent schemas is up to you - JSON Schema, a simple type string, a custom format, etc. Here are example query + type pairings:

| Query | Output Type | Example Output |
|---|---|---|
| "Who is the tenant?" | string | "Greenfield Properties LLC" |
| "When does the lease start?" | date | "2024-03-15" |
| "What is the annual interest rate?" | number | 5.25 |
| "What is the non-compete duration?" | string | "18 months" |
| "List all named parties in the agreement." | array[string] | ["Party A", "Party B", "Party C"] |

| | | |
|---|---|---|
| "List all payment due dates." | array[date] | ["2024-01-01", "2024-04-01", "2024-07-01"] |

## 5. Deliverables

- A Python project with a clear README (setup, usage, design decisions).
- A working CLI or notebook demonstrating end-to-end extraction
- **Test suite and results are mandatory.**
- Share via a private repo link or zip archive.

---

## 6. Bonus (Optional)

- **Confidence Scores:** Return a confidence score (0–1) for each extracted value.
- **Simple UI:** A Streamlit or Gradio interface for upload → query → result.

---